



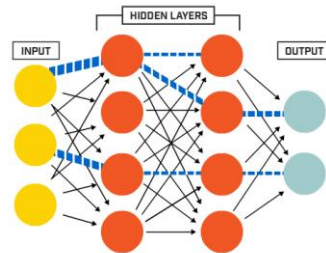
RV College of
Engineering®

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the world

ARTIFICIAL NEURAL NETWORK AND DEEP LEARNING

Deep Reinforcement Learning



Dr Somesh Nandi
Assistant Professor
Dept of AIML
RVCE



Deep Reinforcement Learning

- Herbert Simon's ant hypothesis:

"Human beings, viewed as behaving systems, are quite simple. The apparent complexity of our behavior over time is largely a reflection of the complexity of the environment in which we find ourselves."

- Humans are simple, because they are reward-driven entities.
- All of biological intelligence is owed to this simple fact.
- Reinforcement learning attempts to simplify the learning of complex behaviors by using *reward-driven trial and error*.



Deep Reinforcement Learning

- Human beings are considered simple because they are one-dimensional, selfish, and reward driven entities (when viewed as a whole), and all of biological intelligence is therefore attributable to this simple fact.
- Since the goal of artificial intelligence is to simulate biological intelligence, it is therefore natural to draw inspirations from the successes of biological greed in simplifying the design of highly complex learning algorithms.



Deep Reinforcement Learning

- **A reward-driven trial-and-error process, in which a system learns to interact with a complex environment to achieve rewarding outcomes, is referred to in machine learning parlance as reinforcement learning.**
- In reinforcement learning, the process of trial and error is driven by the need to **maximize the expected rewards over time.**
- Reinforcement learning can be a gateway to the quest for creating truly intelligent agents such as game-playing algorithms, self-driving cars, and even intelligent robots that interact with the environment.



Deep Reinforcement Learning

Example 1 : Atari 2600 is a platform supporting multiple games.

The input to the deep learner from the Atari platform is the display of pixels from the current state of the game.

- The reinforcement learning algorithm predicts the actions based on the display and inputs them into the Atari console.



Deep Reinforcement Learning

Example 2 : AlphaGo : to play the game of Go by using the reward-outcomes in the moves of games drawn from both human and computer self-play.

- Go is a complex game that requires significant human intuition, and the large tree of possibilities (compared to other games like chess) makes it an incredibly difficult candidate for building a game-playing algorithm.
- **AlphaGo** has not only convincingly defeated all top-ranked Go players it has played against , but has contributed to innovations in the style of human play by using unconventional strategies in defeating these players



Deep Reinforcement Learning

Example 3 : Self Driving Cars

- In recent years, deep reinforcement learning has been harnessed in self-driving cars by using the feedback from various sensors around the car to make decisions.
- Although it is more common to use supervised learning (or imitation learning) in self-driving cars, the option of using reinforcement learning is more accurate.
- During the course of driving, these cars now consistently make fewer errors than do human beings.



Deep Reinforcement Learning

Example 4 : Multi armed Bandit problem

Imagine a gambler in a casino with many slot machines (also called "bandits").

- **Each machine gives a reward, but you don't know in advance which machine gives the most rewards.**
- To win as much as possible, you need to:
 - **Explore:** Try different slot machines to figure out which ones pay well.
 - **Exploit:** Keep playing the machine you think gives the best reward.

The Challenge:

- **Exploring new machines might help you learn more, but it could also waste money if the machine doesn't pay well.**
- **Exploiting what you've already learned gives rewards but might miss better options.**
- **The goal is to balance exploration (trying new things) and exploitation (using what you know works).**



Deep Reinforcement Learning

Stateless Algorithms: Multi-Armed Bandits- Solutions

Naïve Algorithm

The gambler tries each slot machine a certain number of times to figure out which one pays the best (this is called exploration).

After testing, they pick the machine they think is best and use it forever (exploitation).



Deep Reinforcement Learning

Stateless Algorithms: Multi-Armed Bandits

Drawbacks :

Hard to Decide When to Stop Testing: How many times should you try each machine to be sure which one is the best? It's not clear.

Takes Too Long: If rewards are rare (like a machine that only gives big wins occasionally), it might take forever to figure out which machine is best.

Wasted Effort: Testing too much wastes time and effort on bad machines.

Big Mistake if Wrong: If you pick the wrong machine, you're stuck with it forever, and you'll keep losing out.



Deep Reinforcement Learning

Stateless Algorithms: Multi-Armed Bandits

Method 2: ϵ -Greedy Algorithm

- The basic idea is to choose a random slot machine for a fraction ϵ of the trials.
- These exploratory trials are also chosen at random (with probability ϵ) from all trials, and are therefore fully interleaved with the exploitation trials.
- In the remaining $(1 - \epsilon)$ fraction of the trials, the slot machine with the best average payoff so far is used.
- Advantage : one is guaranteed to not be trapped in the wrong strategy forever.
- Since the exploitation stage starts early, one is often likely to use the best strategy a large fraction of the time.



Deep Reinforcement Learning

Stateless Algorithms: Multi-Armed Bandits

Method 2: ϵ -Greedy Algorithm

The goal is to balance exploration (trying different slot machines) and exploitation (sticking to the best one).

• Here's how it works:

- **For a small fraction of the time (ϵ , like 10% or 0.1), the gambler picks a random slot machine to try something new (exploration).**
- **For the rest of the time ($1 - \epsilon$, like 90% or 0.9), they stick with the machine that has given the best average reward so far (exploitation).**

Why This Works:

- Random exploration (ϵ fraction) ensures the gambler keeps learning and doesn't get stuck using the wrong machine forever.
- Starting exploitation ($1 - \epsilon$ fraction) early ensures the gambler spends most of the time using the best machine.



Deep Reinforcement Learning

Method 3: Upper Bounding Methods

The upper bounding strategy improves this by combining exploration and exploitation into every trial.

Here's how it works:

1. Optimistic View:

1. Instead of only using the reward (mean) of a slot machine to decide, the gambler adds a "bonus" for uncertainty.
2. This bonus is bigger for slot machines that haven't been tested much.

2. Formula for Decision:

1. The upper bound (U_i) for a slot machine =
Average reward (Q_i) + Confidence bonus (C_i).
So, machines with higher uncertainty (larger bonuses) are more likely to be chosen.



Deep Reinforcement Learning

Stateless Algorithms: Multi-Armed Bandits

Method 3: Upper Bounding Methods

Confidence Bonus (C_i):

- C_i depends on how many times a slot machine has been tried:
 - Rarely tried machines → Higher uncertainty → Larger bonus → More exploration.
 - Frequently tried machines → Lower uncertainty → Smaller bonus → More exploitation.

• Formula for C_i :

$$C_i = K \cdot \frac{\sigma_i}{\sqrt{n_i}}$$

- K : Controls how confident the algorithm is (e.g., higher K = more exploration).
- σ_i : Variation in rewards for the machine.
- n_i : Number of times the machine has been tried.



Stateless vs. State-Based Learning

1.Stateless Algorithms (like bandit algorithms):

1. Each decision (action) is made independently of the past.
2. Past actions only update the agent's knowledge (e.g., which slot machine gives better rewards).
3. The environment itself doesn't change based on past actions.

2.State-Based Learning (in reinforcement learning):

1. Actions not only update the agent's knowledge but also change the environment.
2. The decision depends on the state of the environment, which reflects the result of past actions.



1. Agent and Environment:

Agent: The decision-maker (e.g., a video game player or a self-driving car system).

Environment: The system or setup the agent interacts with (e.g., the video game world or the road).

Actions and States:

1.Action: Any move or decision the agent makes (e.g., pressing a button or steering the car).

2.State: The current situation of the environment, including all relevant variables (e.g., the car's position or the player's location in the game).

Rewards:

1.The environment provides feedback (rewards) based on how well the agent is doing.

2.Rewards aren't always immediate or tied to a single action.

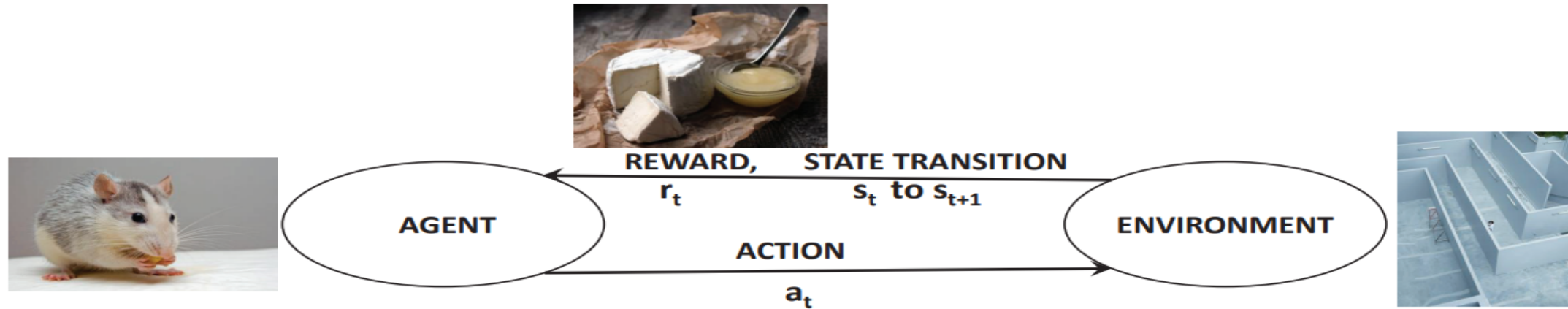
Example:

1.Swerving to avoid a crash in a dangerous situation might give a positive reward.

2.The same swerve in a normal situation might lead to a negative reward.

Deep Reinforcement Learning

The Basic Framework of Reinforcement Learning



1. **AGENT (MOUSE) TAKES AN ACTION a_t (LEFT TURN IN MAZE) FROM STATE (POSITION) s_t**
2. **ENVIRONMENT GIVES MOUSE REWARD r_t (CHEESE/NO CHEESE)**
3. **THE STATE OF AGENT IS CHANGED TO s_{t+1}**
4. **MOUSE'S NEURONS UPDATE SYNAPTIC WEIGHTS BASED ON WHETHER ACTION EARNED CHEESE**

OVERALL: AGENT LEARNS OVER TIME TO TAKE STATE-SENSITIVE ACTIONS THAT EARN REWARDS

- The biological and AI frameworks are similar.
- MDP represented as $s_0 a_0 r_0 s_1 a_1 r_1 \dots s_n a_n r_n$



Basic Framework of Reinforcement

- The entire set of states and actions and rules for transitioning from one state to another is referred to as a Markov decision process.
- The main property of a Markov decision process is that the state at any particular time stamp encodes all the information needed by the environment to make state transitions and assign rewards based on agent actions.
- Finite Number of Actions : Episodic
- Infinite ;Non Episodic



Deep Reinforcement Learning

Examples of Markov Decision Process

- *Game of tic-tac-toe, chess, or Go:* The state is the position of the board at any point, and the actions correspond to the moves made by the agent. The reward is $+1$, 0 , or -1 (depending on win, draw, or loss), *which is received at the end of the game.*
- *Robot locomotion:* The state corresponds to the current configuration of robot joints and its position. The actions correspond to the torques applied to robot joints. The reward at each time stamp is a function of whether the robot stays upright and the amount of forward movement.
- *Self-driving car:* The states correspond to the sensor inputs from the car, and the actions correspond to the steering, acceleration, and braking choices. The reward is a function of car progress and safety.



Deep Reinforcement Learning

Challenges in Reinforcement Learning

1. When a reward is received (e.g., winning a game of chess), it is not exactly known how much each action has contributed to that reward.
 - This problem is referred to as the **credit-assignment problem**.

2. The reinforcement learning system might have a very **large number of states**, and must be able to make sensible decisions in states it has not seen before.

This task of model generalization is the primary function of deep learning



Deep Reinforcement Learning

Challenges in Reinforcement Learning

3. A specific choice of action affects the collected data in regard to future actions.

- **Trade-off between exploration and exploitation.** If actions are taken only **to learn their reward**, then it incurs a cost to the player. On the other hand, sticking to known actions might result in suboptimal decisions.

4. Reinforcement learning merges the notion of data collection with learning.

- Realistic simulations of large physical systems such as **robots and self-driving cars** are limited by the need to physically perform these tasks and gather responses to actions in the presence of the practical dangers of failures.



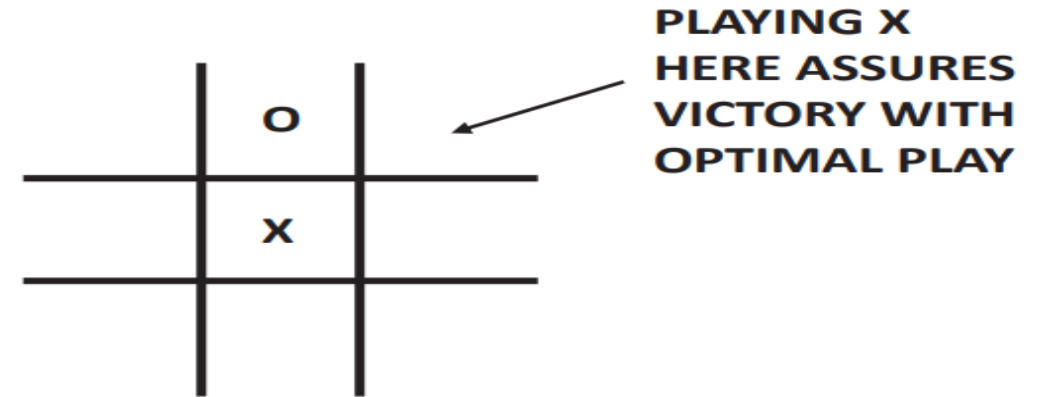
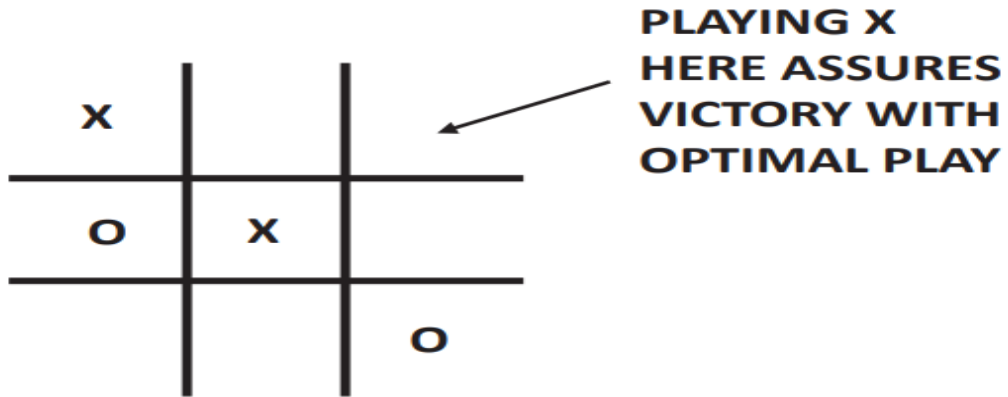
Deep Reinforcement Learning

Challenges in Reinforcement Learning

The inability to gather sufficient data in real settings beyond simulated and game-centric environments is arguably the single largest challenge to reinforcement learning.

Deep Reinforcement Learning

Tic-Tac-Toe



- **Traditional reinforcement learning:** Learn through trial-and-error the *long-term* value of each state.
- Long-term values are not the same as rewards.
 - Rewards not realized immediately because of stochasticity (e.g., slot machine) or delay (board game).



Reinforcement Learning for Tic-Tac-Toe

- **Key Steps in Learning Tic-Tac-Toe:**

1. **State and Action Representation:**

1. Every **board position** is a **state**.
2. Every valid move (placing 'X' or 'O') is an **action**.

2. **Number of States:**

1. The 3×3 tic-tac-toe board has at most **19,683 states** (3 possibilities for each of the 9 positions: 'X', 'O', or blank).

3. **Learning State-Action Values:**

1. The algorithm keeps track of the value of each **state-action pair (s, a)**:
 1. How good is it to play action **a** in state **s**?
2. These values are learned based on how actions performed historically.



- What is a Reward?
- A reward is the score or feedback given after a game:
- Win: Positive reward (e.g., +1).
- Loss: Negative reward (e.g., -1).
- Draw: No reward (e.g., 0).
- It tells the algorithm how good or bad a move was in achieving the game's goal.



Deep Reinforcement Learning

Tic-Tac-Toe

- Main difference from multi-armed bandits is that we need to learn the long-term rewards for each action in each *state*.
- An eventual victory earns a reward from $\{+1, 0, -1\}$ with delay.
- A move occurring r moves earlier than the game termination earns *discounted* rewards of $\{\gamma^{r-1}, 0, -\gamma^{r-1}\}$.
 - Future rewards would be less certain in a replay.
- Assume that a fixed pool of humans is available as opponents to train the system (self-play possible).

- **Discounting** means reducing the importance of a reward as we go further back in the game.
- Earlier moves are more important because they set the foundation for the result.
- This is done using a **discount factor γ** , where $0 < \gamma < 1$

$$\text{Reward for that move} = \text{Game Result} \times \gamma^{(r-1)}$$

Actions made closer to the end (e.g., the last move) are directly responsible for the result, so they get a higher reward.

Actions made earlier are less directly connected, but they still contribute, so they get smaller rewards.

Let's say:

- $\gamma = 0.9$
- The game ends in 5 moves, and you win (+1 reward).

Rewards for Each Move:

1. Move 1 (5 steps before the end):

$$\text{Reward} = +1 \times (0.9^4) = +0.6561$$

2. Move 2 (4 steps before the end):

$$\text{Reward} = +1 \times (0.9^3) = +0.729$$

3. Move 3 (3 steps before the end):

$$\text{Reward} = +1 \times (0.9^2) = +0.81$$

4. Move 4 (2 steps before the end):

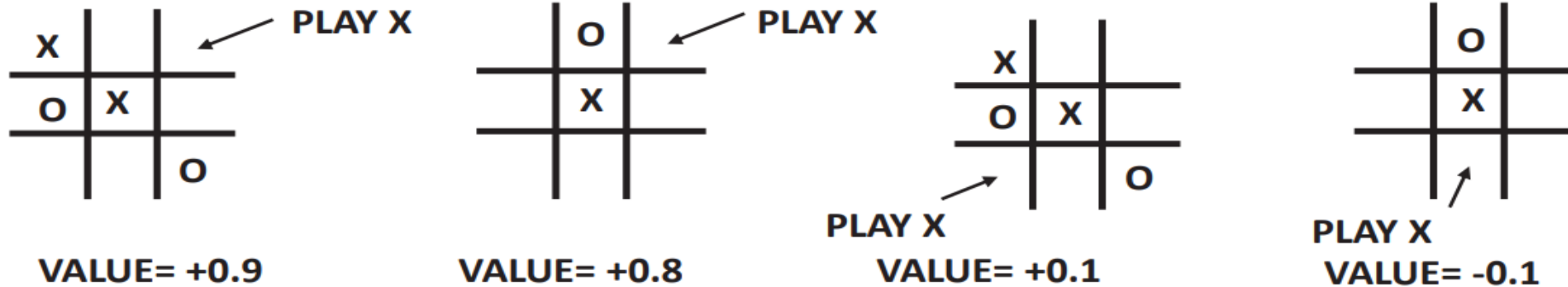
$$\text{Reward} = +1 \times (0.9^1) = +0.9$$

5. Move 5 (last move):

$$\text{Reward} = +1 \times (0.9^0) = +1$$

Deep Reinforcement Learning

At the End of Training



- Typical examples of normalized values of moves
- ϵ -greedy will learn the strategic values of moves.
- Rather than state-action-value triplets, we can equivalently learn state-value pairs.



Deep Reinforcement Learning

Strawman ϵ -Greedy Algorithm with Deep Learning for Chess [Primitive: Don't Try It!]

- Convolutional neural network takes board position as input and produces position value as output.
- Use ϵ -greedy algorithm on output values to simulate a full game.
- **After game of X moves:** Create X training points with board position as input feature map and targets from $\{\gamma^{r-1}, 0, -\gamma^{r-1}\}$ depending on move number and win/loss.
- Update neural network with these X training points.
- Repeat the steps above.



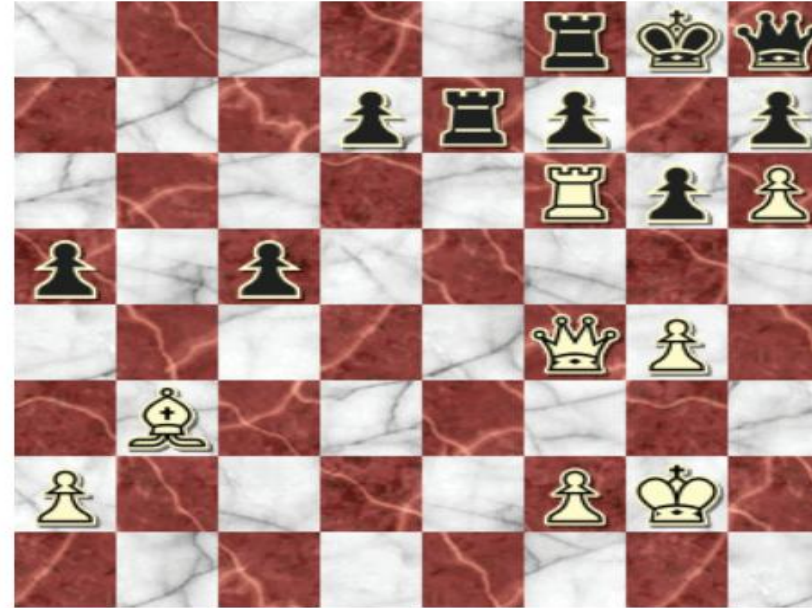
Deep Reinforcement Learning

Reinforcement Learning in Chess and Go

- The reinforcement learning systems, *AlphaGo* and *Alpha Zero*, have been designed for chess, Go, and shogi.
- Combines various advanced deep learning methods and Monte Carlo tree search.
- Plays positionally and sometimes makes sacrifices (much like a human).
 - Neural network encodes evaluation function learned from trial and error.
 - More complex and subtle than hand-crafted evaluation functions by conventional chess software.

Deep Reinforcement Learning

Examples of Two Positions from Alpha Zero Games vs Stockfish



- Generalize to unseen states in training.
- Deep learner can recognize subtle positional factors because of trial-and-error experience with feature engineering.



Straw Man Algorithm in Reinforcement Learning

- **A Straw Man Algorithm is a simple, naive baseline used for comparison in reinforcement learning.**
- **It's often implemented to test basic functionality or provide a benchmark for evaluating more sophisticated algorithms.**
- **While not meant to be optimal, it gives an idea of how a basic strategy performs in a given environment.**



Straw Man Algorithm in Reinforcement Learning

- A 5x5 grid where: The agent starts at the bottom-left corner.
- The goal is at the top-right corner.
- Moving into a wall keeps the agent in the same position.
- Rewards:
 - Reaching the goal: +10.
 - Every step: -1 (penalty for taking too long).

Straw Man Algorithm:

1.Policy: Choose a random action (e.g., UP, DOWN, LEFT, RIGHT) at every step, regardless of the current state.

2.No Learning: The agent doesn't update its behavior based on rewards or outcomes.

3.Termination: Stops when the agent reaches the goal or a maximum number of steps is taken.



Deep Reinforcement Learning

Other Challenges

- Chess and tic-tac-toe are *episodic*, with a maximum length to the game (9 for tic-tac-toe and ≈ 6000 for chess).
- The ϵ -greedy algorithm updates episode by episode.
- What about infinite Markov decision processes like robots or long episodes?
 - Rewards received continuously.
 - Not optimal to update episode-by-episode.
- Value function and Q-function learning can update after each *step* with *Bellman's equations*.



Boot Strapping for Value Function Learning

- **Bootstrapping** is a technique in reinforcement learning (RL) where an agent updates its knowledge or value estimates incrementally during the learning process, using estimates of future values rather than waiting for the actual outcome.
- Bootstrapping is essential for efficient learning in environments where the agent has to deal with **long-term decisions, large state spaces, or tasks that may continue indefinitely.**

The process of **using current progress to estimate future success** is exactly what bootstrapping



- **Goal:** Ride the bike for 10 meters without falling.
- **Actions:** Pedaling, balancing, and steering.
- **Reward:** The distance you manage to ride before falling.

-

How Bootstrapping Works:

1. Initial Attempts:

On your first try, you wobble and only manage to ride 2 meters before falling. You learn that balancing is important.

Your brain thinks, "If I keep practicing, I can probably ride further."

2. Second Attempt:

This time, you balance better and ride 4 meters. You adjust your steering based on what worked in the first attempt.

You estimate, "If I keep improving like this, I might reach 10 meters soon!"

3. Ongoing Learning:

After every attempt, you update your expectations (value function) based on how far you rode and how much better you think you can get.

Instead of waiting to ride the full 10 meters to learn, you build your confidence and strategy step by step, improving as you go.



Deep Reinforcement Learning

An Infinite Markov Decision Process

- Sequence below is of infinite length (continuous process)

$$s_0 a_0 r_0 s_1 a_1 r_1 \dots s_t a_t r_t \dots$$

- The cumulative reward R_t at time t is given by the discounted sum of the immediate rewards for $\gamma \in (0, 1)$:

$$R_t = r_t + \gamma \cdot r_{t+1} + \gamma^2 \cdot r_{t+2} + \gamma^3 \cdot r_{t+3} \dots = \sum_{i=0}^{\infty} \gamma^i r_{t+i} \quad (1)$$

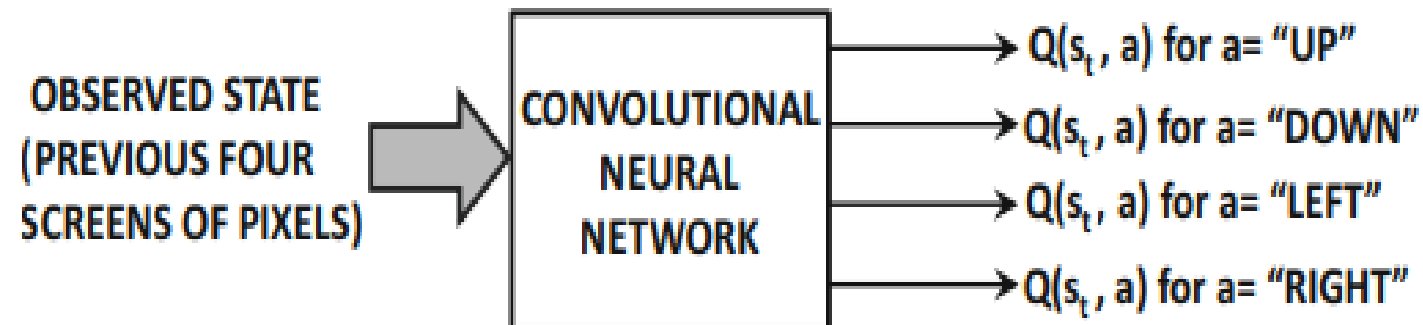
- Future rewards worth less than immediate rewards ($\gamma < 1$).
- Choosing $\gamma < 1$ is not essential for episodic processes but critical for long MDPs.

The Q-function or Q-value $Q(s_t, a_t)$ represents the expected cumulative reward an agent can achieve by taking a specific action a_t in a given state s_t and following a specific policy thereafter.

Deep Reinforcement Learning

Deep Learning as Functional Approximator

- **Consider the Atari setting in which a fixed window of the last few snapshots of pixels provides the state s_t .**
- **Assume that the feature representation of s_t is denoted by X_t . The neural network uses X_t as the input and outputs $Q(s_t, a)$ for each possible legal action a from the universe of actions denoted by the set A .**





Deep Reinforcement Learning

Deep Learning as Functional Approximator

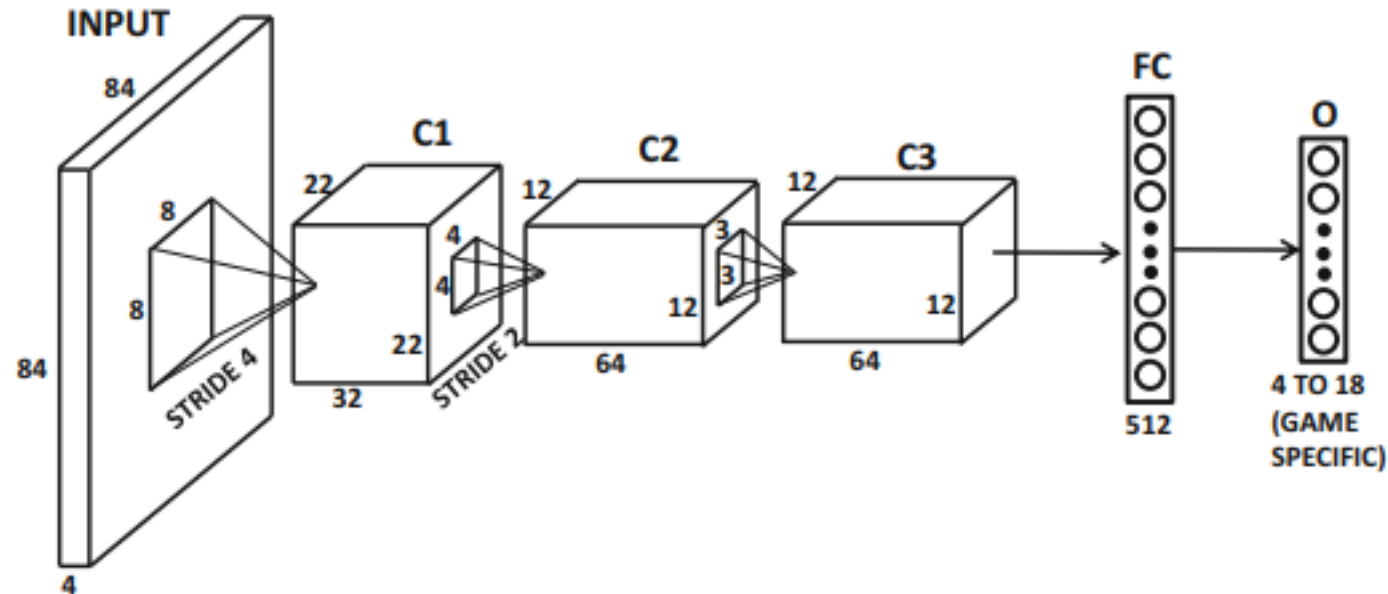
Assume that the neural network is parameterized by the vector of weights W , and it has $|A|$ outputs containing the Q-values corresponding to the various actions in A .

In other words, for each action $a \in A$, the neural network is able to compute the function $F(X_t, W, a)$, which is defined to be the learned estimate of $Q(s_t, a)$:

$$F(\bar{X}_t, \bar{W}, a) = \hat{Q}(s_t, a)$$

Deep Reinforcement Learning

Example: Neural Network for Atari Setting





On-Policy Versus Off-Policy Methods: SARSA

- A **policy** is the agent's "strategy" or "rulebook" for deciding what action to take in any given situation.
- **On-Policy Methods:**
- The agent learns about the policy it is currently following.
- It updates its strategy (policy) based on the actions it is actively using to explore the environment.
- **Key Idea:** “Learn by doing
- Example:
- Imagine learning to ride a bicycle. You're figuring it out as you go—falling, adjusting your balance, and improving. This is **on-policy** learning because you're learning while actively using your own approach.



On-Policy Versus Off-Policy Methods: SARSA

- The agent learns about a policy **different from the one it is currently using** to explore.
- It might take random actions (to explore better), but it's learning how to behave if it followed a smarter, more optimized strategy.
- **Key Idea: “Learn by watching or experimenting.”**
- Example:
- Imagine watching YouTube tutorials on riding a bicycle and practicing based on those tips. Here, you're **off-policy** because you're learning about the "ideal" way to ride, even if your current attempts are messy.



On-Policy Versus Off-Policy Methods: SARSA

- **Q-Learning** is a type of **off-policy** method, while its counterpart, **SARSA**, is an **on-policy** method.
- Here's how they differ:
 1. **Q-Learning (Off-Policy):**
 1. Q-Learning doesn't care what the agent is currently doing (its current policy).
 2. It learns the **best possible policy** by always assuming the agent will pick the best action in the future.
 3. **Example:** Watching someone else's mistakes or successes and learning from them to create an ideal strategy.
 2. **SARSA (On-Policy):**
 1. SARSA updates its knowledge based on the actions the agent is actually taking (its current policy).
 2. It focuses on improving the policy the agent is currently using.
 3. **Example:** Learning while riding your bicycle and improving your balance.



On-Policy Versus Off-Policy Methods: SARSA

- Mathematical Representation

SARSA updates the Q-value based on the **current policy** the agent is using.

The update rule:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

Explanation:

- $Q(s_{t+1}, a_{t+1})$: The Q-value for the next state s_{t+1} and the next action a_{t+1} , based on the agent's current policy.
- The update depends on the actual actions the agent takes under its current strategy.

2. Off-Policy Method: Q-Learning

Q-Learning updates the Q-value assuming the **best possible policy** (greedy) is followed in the future, regardless of the current policy.

The update rule:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

Explanation:

- $\max_{a'} Q(s_{t+1}, a')$: The maximum Q-value of all possible actions a' in the next state s_{t+1} .
- The agent assumes it will act optimally in the future, even if it's currently exploring or making random moves.



Modeling States Versus State-Action Pairs

State:

- A state is like the current "situation" or "position" you are in.
- Example: Imagine you are playing chess. The current arrangement of the chess pieces is your state.

•State-Action Pair:

- A **state-action pair** is when you combine a situation (state) with a decision (action).
- Example: In the same chess game, the state could be the chessboard setup, and the action could be "move the queen to D4."



Modeling States Versus State-Action Pairs

1. Modeling States (Value Function $V(s)$):

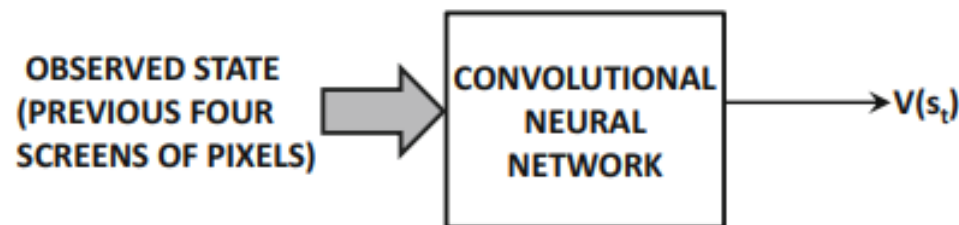
1. Think of this as just rating each situation (state).
2. Example:
 1. If you're in a chess state where your pieces are strong, you might give it a value of **80 points**.
 2. A bad state where you're losing might get only **20 points**.
3. You just ask: "**How good is this situation for me?**"

2. Modeling State-Action Pairs (Q-Function $Q(s,a)$):

1. Here, you think about both the situation and the actions you can take.
2. Example:
 1. In the same chess state, you consider different actions:
 1. Moving the queen to D4 might give a value of **50 points**.
 2. Moving a pawn forward might give **30 points**.
3. You ask: "**How good is it to take this specific action in this situation?**"

Deep Reinforcement Learning

Modeling States Versus State-Action Pairs



- Instead of outputting values of state-action pairs we can output just values.
- Q-Learning and SARSA can be implemented with this architecture as well.
 - General class of temporal difference learning $\Rightarrow$ Multi-step bootstrapping
 - Can explore a forward-looking tree for arbitrary bootstrapping.



Deep Reinforcement Learning

Modeling States Versus State-Action Pairs

Let the value of the state s_t be denoted by $V(s_t)$. Now assume that you have a parameterized neural network that uses the observed attributes $\bar{X}_t$ (e.g., pixels of last four screens in Atari game) of state s_t to estimate $V(s_t)$. An example of this neural network is shown in Figure 9.5. Then, if the function computed by the neural network is $G(\bar{X}_t, \bar{W})$ with parameter vector $\bar{W}$, we have the following:

$$G(\bar{X}_t, \bar{W}) = \hat{V}(s_t) \quad (9.13)$$

we will assume that we have some reasonable heuristic policy for choosing the actions that uses the forward-looking state values in some way.

For example, if we evaluate each forward state resulting from an action and select one of them based on a pre-defined policy (e.g., ϵ -greedy), the approach is the same as SARSA.



Deep Reinforcement Learning

Modeling States Versus State-Action Pairs

Therefore, the bootstrapped ground-truth estimate for $V(s_t)$ can be obtained with the help of this lookahead:

$$V(s_t) = r_t + \gamma V(s_{t+1}) \quad (9.14)$$

This estimate can also be stated in terms of the neural network parameters:

$$G(\bar{X}_t, \bar{W}) = r_t + \gamma G(\bar{X}_{t+1}, \bar{W}) \quad (9.15)$$



Deep Reinforcement Learning

Monte Carlo Tree Search

- Monte Carlo Tree Search (MCTS) is a smart way for computers to make decisions, especially in games or problems where there are many possible moves and outcomes.

How Does MCTS Work?

MCTS is like exploring a decision tree where:

- Each **node** represents a possible situation (state).
 - Each **branch** represents an action you can take to move to the next state.
-
- Selection
 - Expansion
 - Simulation
 - BackPropagation



Deep Reinforcement Learning

Let's imagine you're in a huge toy store, and you want to find the most fun toy. You don't have time to play with all the toys, so you need a plan.

Selection:

- Start from the current situation (root node).
- Pick the most promising child node (action) to explore, using a strategy like **UCB1** (Upper Confidence Bound for Trees).

•Example

- You start by looking at a shelf full of toys.
- You pick the toy that seems most fun based on what you've tried before.
- Example: If a robot toy was fun before, you might choose a similar one.



Deep Reinforcement Learning

- **Expansion:**
 - If the chosen node isn't fully explored, create a new child node.
 - This represents trying a new action.
- **If you see a new toy on the shelf, you decide to try it out.**
- **Example: You've never played with a remote-controlled car, so you give it a chance.**
- **Simulation:**
 - From the new node, simulate a random sequence of actions until the game ends (or a predefined point).

You take the toy and play with it for a little while.

Example: You race the remote-controlled car around the store to see if it's fun.



Deep Reinforcement Learning

Backpropagation:

- Use the result of the simulation to update the value of the nodes that were part of the path.
- If a simulation ends in a win, all nodes along the path to that win become more favorable.

Backpropagation:

- After playing, you decide how much fun it was and remember it.
- If it was super fun, you'll want to play with it again. If not, you'll avoid it next time.